Guía de TypeScript para Estudiantes

Manual de Referencia Rápida con Conceptos Clave, Ejemplos de Código y Ejercicios Prácticos

Esta guía reúne los conceptos fundamentales necesarios para comenzar a programar en TypeScript. Aprenderás a utilizar variables, tipos de datos, operadores, estructuras de control, objetos, arreglos y funciones mediante ejemplos claros y directos basados en casos prácticos.

1. Variables y Tipos de Datos

Una variable permite guardar un valor en memoria para utilizarlo más adelante en el programa. En TypeScript se utilizan principalmente dos palabras clave para declarar variables: **const** y **let**.

- **const:** se utiliza cuando el valor asignado no cambiará ni se reasignará posteriormente.
- **let:** se emplea cuando la variable necesita recibir otro valor más adelante durante la ejecución.

```
const nombre: string = "Ana";
let edad: number = 20;

edad = 21; // Permitido porque se declaró con let
```

Tipos de Datos Básicos

Los dos puntos (**:**) indican explícitamente el tipo de dato que almacenará la variable. Los tipos fundamentales son:

Tipo	Descripción	Ejemplo de Código
string	Cadenas de texto	`const saludo: string = "Hola";`
number	Números enteros o decimales	`const cantidad: number = 5;`
boolean	Valores lógicos (true / false)	`const esEstudiante: boolean = true;`

Inferencia de Tipos

TypeScript posee la capacidad de deducir automáticamente el tipo de una variable a partir del valor inicial que se le asigna, sin necesidad de escribirlo explícitamente:

```
const ciudad = "Tucumán"; // TypeScript deduce automáticamente que es de tipo string
```

> **RECOMENDACIÓN BÁSICA**
> Usa 'const' por defecto para evitar reasignaciones accidentales. Cambia a 'let' únicamente cuando sepas con certeza que el valor deberá modificarse más adelante en el programa.

2. Operadores de Comparación y Lógicos

Los operadores permiten comparar valores y evaluar condiciones en el código. Las comparaciones siempre producen un resultado booleano (**true** o **false**).

```
const edad = 20;

const esMayor = edad >= 18;          // true
const tieneEdadExacta = edad === 20; // true
const noTieneDiez = edad !== 10;     // true
```

Principales Operadores de Comparación

Operador	Significado y Uso
`===`	Igualdad estricta (compara valor y tipo)
`!==`	Desigualdad estricta
`> y <`	Mayor que y menor que
`>= y <=`	Mayor o igual que, menor o igual que

Operadores Lógicos y Ternario

Para combinar múltiples condiciones se utilizan los operadores lógicos:

- **&& (AND):** ambas condiciones deben ser verdaderas para que el resultado sea verdadero.
- **|| (OR):** alcanza con que al menos una de las condiciones sea verdadera.
- **! (NOT):** invierte un valor booleano (true pasa a false y viceversa).

```
const tieneEntrada = true;
const esMayor = edad >= 18;

const puedeIngresar = esMayor && tieneEntrada; // Evalúa a true sólo si ambas se
```

```
cumplen
```

El **operador ternario** permite seleccionar entre dos valores según una condición:

```
const mensaje = edad >= 18 ? "Puede ingresar" : "No puede ingresar";
```

> **REGLA DE LECTURA**
> El operador ternario se lee: 'si la condición es verdadera, se usa el valor antes de los dos puntos (:); si es falsa, se usa el de después'.

3. Objetos, Interfaces y Enumeraciones

Un **objeto** agrupa información relacionada mediante propiedades (clave-valor). Puede contener propiedades simples o incluso otros objetos anidados:

```
const persona = {
  nombre: "Ana",
  edad: 20,
  vive: true,
};

// Acceso a propiedades
console.log(persona.nombre); // "Ana"
console.log(persona["edad"]); // 20

// Objeto anidado
const estudiante = {
  nombre: "Luis",
  domicilio: {
    calle: "San Martín",
    numero: 123,
  },
};
```

Interfaces

Una **interface** define la forma exacta que debe tener un objeto. Es fundamental para dejar claros los nombres y tipos de sus propiedades:

```
interface Persona {
  nombre: string;
  edad: number;
  tieneAcompanante: boolean;
}

const visitante: Persona = {
  nombre: "Micaela",
  edad: 16,
  tieneAcompanante: true,
};
```

Enumeraciones (Enums)

Una **enum** representa un conjunto de opciones conocidas y restringidas. Evita escribir textos arbitrarios para opciones que solo admiten alternativas fijas:

```
enum Nacionalidad {
  AR = "AR",
  BR = "BR",
  CL = "CL",
}

interface PersonaConNacionalidad {
  nombre: string;
  nacionalidad: Nacionalidad;
}

const persona: PersonaConNacionalidad = {
  nombre: "Ana",
  nacionalidad: Nacionalidad.AR,
};
```

4. Estructuras Condicionales

Permiten ejecutar distintas instrucciones en el código según se cumpla o no una condición.

Estructura if, else if y else

Las condiciones se revisan en estricto orden. Cuando una resulta verdadera, se ejecuta su bloque y se omiten los siguientes:

```
const edad = 16;
const tieneAcompanante = true;

if (edad >= 18) {
  console.log("Puede ingresar");
} else if (edad >= 12 && tieneAcompanante) {
  console.log("Puede ingresar con permiso especial");
} else {
  console.log("No puede ingresar");
}
```

Estructura switch

La instrucción **switch** es una alternativa conveniente cuando se compara un mismo valor con varias alternativas distintas:

```
const dia = "martes";

switch (dia) {
  case "lunes":
    console.log("Comienza la semana");
    break;
  case "martes":
    console.log("Segundo día");
    break;
  default:
    console.log("Otro día");
}
```

> **REGLA DE SWITCH**
> La instrucción 'break' evita que la ejecución continúe en el caso siguiente. El bloque 'default' se ejecuta si no coincide ningún caso previo.

5. Arreglos y Ciclos

Un **arreglo** guarda varios valores de manera ordenada. En TypeScript, el tipo de un arreglo se escribe como **tipo[]**. Las posiciones de sus elementos comienzan siempre en cero (0):

```
const edades: number[] = [18, 20, 25];
const nombres: string[] = ["Ana", "Luis"];

console.log(nombres[0]);    // "Ana"
console.log(edades.length); // Cantidad total de elementos
```

Recorrido con Ciclo for

Un ciclo **for** permite repetir instrucciones. Es común usar el índice **i** para recorrer un arreglo paso a paso:

```
const puntajes: number[] = [7, 4, 9];

for (let i = 0; i < puntajes.length; i++) {
  console.log(puntajes[i]);
}
```

Conteo de Elementos con Condicional

Para contar elementos que cumplen una condición específica dentro de un arreglo, se utiliza una variable contador:

```
const numeros: number[] = [1, 12, 8, 20];
let contador = 0;

for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] <= 10) {
    contador++;
  }
}

console.log(contador); // Muestra 2
```

Método push()

El método **push()** agrega un nuevo elemento al final del arreglo:

```
const colores: string[] = ["azul", "rojo"];
colores.push("verde");
```

6. Funciones

Una **función** agrupa instrucciones reutilizables. Puede recibir parámetros tipados y devolver un resultado con un tipo específico:

```
function sumarCinco(numero: number): number {
  return numero + 5;
}

const resultado = sumarCinco(10);
console.log(resultado); // 15
```

Funciones con Arreglos

Una función también puede recibir arreglos como parámetros y devolver nuevos arreglos procesados:

```
function filtrarMayores(numeros: number[], limite: number): number[] {
  const resultado: number[] = [];

  for (let i = 0; i < numeros.length; i++) {
    if (numeros[i] > limite) {
      resultado.push(numeros[i]);
    }
  }

  return resultado;
}

console.log(filtrarMayores([10, 20, 30], 15)); // Devuelve [20, 30]
```

Funciones sin Retorno (void)

Si una función no necesita devolver ningún valor (por ejemplo, si sólo imprime en pantalla), se declara el retorno como **void**:

```
function saludar(nombre: string): void {
  console.log(`Hola, ${nombre}`);
}
```

7. Sección Práctica para Estudiantes

Pon a prueba lo aprendido resolviendo los siguientes ejercicios:

N°	Consigna del Ejercicio	Conceptos Involucrados
1	Declarar una variable con tu nombre y otra con tu edad.	Uso de const / let y tipos string / number
2	Crear una condición que indique si una persona es mayor de edad.	Operadores de comparación y estructura if/else
3	Definir una interfaz Producto con nombre, precio y disponible.	Interfaces con string, number y boolean
4	Recorrer un arreglo de números y contar cuántos son mayores que 10.	Arreglos number[], ciclo for, condicional e incremento
5	Crear una función que reciba un arreglo de nombres y devuelva cuántos contiene.	Funciones con parámetros de tipo arreglo y uso de .length